BENCHMARK OPERATIONS

What a governed call costs

QueryAPIGate 0.42 on PostgreSQL, measured properly: masking and table allow-lists now cost about 7%, one worker serves 730-875 calls a second, and most of what's left is round trips. The full method, every result with its range, where the time goes, a sizing rule, and what our first two attempts got wrong.

24 October 2026 · 23 min read

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?p=what-a-governed-call-costs. This copy was made on 4 November 2026.

A data gateway puts something between your callers and your database: identity, limits, grants, a SQL guard, masking, an audit record. The fair question is what that costs on every call. For 0.42 we measured it on PostgreSQL, had the method reviewed twice, fixed what the reviews found, and published the harness so you can check our numbers. This post is the whole of it.

The short answer

One gunicorn worker, as in the Docker image, serving a saved query by primary key from pgbench's 10-million-row `pgbench_accounts` table, 32 callers at once:

	0.41	0.42	0.42 WITH THE METADATA CACHE
Calls a second	682	743	875
One call on its own (p50)	1.43 ms	1.38 ms	1.05 ms
Calls a second with masking and a table allow-list	171	693	802

- **Governance is now close to free:** a key with column masking and `allowed_tables` costs about 7% more than one without. In 0.41 it was four times slower.
- **One worker serves 730-875 calls a second.** p99 holds at about 4 ms up to 75% of that, then climbs.
- **Processes add throughput:** eight workers served 2,165-2,727 calls a second on four physical cores.
- **PostgreSQL is never the limit here.** At most it used 2 of the 4 hardware threads it had.
- **What's left of a call** is about a millisecond of Python, a fifth of it logging, plus round trips to PostgreSQL. A call in 0.42 makes 7 of them, 3 to QueryAPIGate's own store and 4 to the data database. With the metadata cache it makes 4. Across a real network, those round trips are most of the cost.

How to read the numbers. Every comparison below is the median of 5 interleaved rounds, with its min-max range in brackets. A difference counts as real only when the two ranges don't overlap. The cold-path test and the profile are single long runs, and they're marked where they appear.

How we measured

The machine, kept steady

One laptop: an Intel i7-12700H with 6 performance cores (CPUs 0–11, two threads each) and 8 efficiency cores (CPUs 12–19), 32 GB of memory, Ubuntu with kernel 6.8. A laptop isn't a server, so most of the method is about keeping it from changing under us:

- **AC power, and the `performance` power profile and CPU energy preference.** Turbo stays on, since turning it off needs root; its effect is measured below.
- **Every part on its own cores,** so none takes CPU from another:

PART	CPUS	HOW
QueryAPIGate (gunicorn)	0–7: four performance cores, both threads of each	`taskset -c 0-7`
PostgreSQL	8–11: two performance cores	`docker run --cpuset-cpus 8-11`
Load generator (oha) and pgbench	12–19: the efficiency cores	`docker run --cpuset-cpus 12-19`

- **No proxy in the path.** The gateway reaches PostgreSQL at the container's own address, not through Docker's published port, whose userland proxy is one more unpinned process copying every byte.
- **A floor around every block:** 10 seconds of `/health` on one worker, before and after each block, as a check that the machine hasn't changed speed. It's the same work every time. Both sessions sagged about 10% in their last hour, after 90 minutes of load. That's why every table compares builds only within its own block, where they take turns.
- **The load generator was never the bottleneck.** It used at most 0.79 of its 8 cores, at 2,700 calls a second.

The results come from two sessions on that machine, with the same harness and settings:

SESSION	WHEN	WHAT	`/HEALTH` FLOOR
A	2026-10-23, 21:00–23:24	pgbench, throughput, single-call latency, cold path, access log, profile	1,934–2,296 a second
B	2026-10-24, 08:45–11:08	latency under load, scale-out, network, mixed and hot rows, threads, round trips	1,959–2,258 a second

Where the two sessions overlap, they agree: one worker at 32 callers served 743 lookups a second in session A, and 730 in session B.

Turbo, measured

The busiest gateway core's clock was sampled every two seconds of every run:

- **Overall, 2.4 to 4.4 GHz.** That range is between blocks: with eight busy workers, the cores run at a lower all-core turbo.

- **Within a cell - the same build, call and setting - only ±1%.**
- **The rare fast outlier rounds are clock spikes.** One 0.41 round ran at 4.09 GHz against its cell's usual 3.65, and served 11% more.

So one turbo burst can make one round look good. That's why there are 5 rounds, and why the rule looks at ranges: an outlier widens a range instead of moving a median.

When a difference counts

Builds take turns in an order that rotates each round - A B C, then B C A, then C A B - so none always runs first, warmest or coolest. A difference counts only when the two ranges don't overlap.

How often would that rule call a difference real when there isn't one? If two builds were really the same, every ordering of their 2n results would be equally likely, and the ranges would come out clear in 2 of the C(2n, n) orderings. For 5 rounds, that's 2 in 252, under 1%. For 3 rounds it's 2 in 20, which is 10% - too likely, which is why our second attempt's 3-round blocks were all rerun with 5. Across the roughly 40 comparisons here, perhaps one of the narrow ones is luck. The big ones aren't.

The builds

LABEL	WHAT
0.41	the 0.41.0 release (`f61d4ed`)
0.42	the code that shipped as 0.42.0 (`c9b15c2` in session A and `99db30b` in session B, with the same code - only docs changed between them)
Cached	0.42 with `QUERYAPIGATE_METADATA_CACHE_TTL=5`

Each build is exported from git and run as the Docker image runs it: gunicorn's `gthread` worker, 8 threads, access log on, and every other setting at its default. That means 0.41 has a pool of 5 and 0.42 a pool of 8.

The software: Python 3.12.3, gunicorn 26.2.0, Flask 3.1.1, psycopg2 2.9.10, sqlglot 25.34.1, PostgreSQL 16.15, and oha 1.16.0, pinned by image digest.

The database

pgbench's TPC-B-like schema at scale 100 (`pgbench -i -s 100`), 1.5 GB, entirely in memory (`shared_buffers=2GB` , `max_connections=300` , `synchronous_commit=on`):

TABLE	ROWS	COLUMNS	KEY
`pgbench_accounts`	10,000,000	`aid integer`, `bid integer`, `abalance integer`, `filler character(84)`	primary key `aid`
`pgbench_branches`	100	`bid integer`, `bbalance integer`, `filler character(88)`	primary key `bid`
`pgbench_tellers`	1,000	`tid integer`, `bid integer`, `tbalance integer`, `filler character(84)`	primary key `tid`
`pgbench_history`	0	`tid`, `bid`, `aid`, `delta integer`, `mtime timestamp`, `filler character(22)`	none

QueryAPIGate's own store - keys, saved queries, connections, run history - is a second database in the same PostgreSQL, as a production deployment would have it.

The calls and the keys

Three saved queries, each called with a random id on every request so nothing is answered from a warm cache:

SAVED QUERY	SQL	CALLED AS
`account`, a **lookup**	`SELECT aid, bid, abalance FROM pgbench_accounts WHERE aid = :aid`	`/q/account?aid=` 1 to 9,999,999
`account_page`, a **100-row page**	`SELECT a.aid, a.abalance, b.bid, b.bbalance FROM pgbench_accounts a JOIN pgbench_branches b ON b.bid = a.bid WHERE a.aid BETWEEN :start AND :start + 99 ORDER BY a.aid`	`/q/account_page?start=` 1 to 999,999 `&page_size=100`
`deposit`, a **write**	`UPDATE pgbench_accounts SET abalance = abalance + :delta WHERE aid = :aid`	`/q/deposit?aid=` 1 to 9,999,999 `&delta=` 1 to 9

And three keys:

- **plain:** `account` and `account_page`, and nothing else;
- **governed:** the same, plus `allowed_tables` (`pgbench_accounts`, `pgbench_branches`) and a masking rule that hashes `*.abalance`;
- **writer:** `deposit` only, with `allow_writes` and `allowed_write_ops: ["update"]`.

Across both sessions, all 12 million requests that completed returned 200. Requests still in flight when a run's time was up aren't counted either way.

The load

- **Closed loop, for throughput.** `oha -c N` keeps N callers busy, each sending its next request when the last one answers: 30 seconds per run, after a 5-second warm-up that fills the pool and the governance caches.

- **Open loop, for latency under load.** `oha -q R --latency-correction` sends R requests a second whatever happens, and times each from when it *should* have gone out. A closed loop under-reports latency at saturation, because a slow answer delays the next request; this avoids that, which is called coordinated omission. The rates run from 25% to 110% of capacity, 20 seconds each, and capacity is measured first in the same block.

What every run records

- **Load:** requests a second, the success rate and every status code; p50, p90, p99 and p99.9.
- **CPU, in cores:** the gateway's (from `/proc`), PostgreSQL's (from its container's cgroup), and the load generator's cores (from `/proc/stat`).
- **Transactions per request**, on the store and on the data database.
- **The machine:** the busiest core's clock, and the CPU package temperature, which ran from 65 to 100 °C.

All of it is in the repository: one JSON line per run, and every table generated from them.

Results

The database on its own

pgbench, from the load generator's cores:

WORKLOAD	CLIENTS	TRANSACTIONS A SECOND	AVERAGE LATENCY
Primary-key read	1	38,289 (37,465-39,586)	0.026 ms
Primary-key read	32	112,321 (96,140-123,670)	0.285 ms
TPC-B: three updates, a select, an insert	32	12,970 (12,617-15,293)	2.5 ms

A lookup costs PostgreSQL 0.03 ms. Everything a call through QueryAPIGate takes beyond that is the gateway and its round trips.

One worker, 32 callers

CALL		0.41	0.42	CACHED
Lookup, plain key	calls/s	682 (679–686)	743 (736–746)	875 (869–876)
	p99	55.8 ms	54.1 ms	45.1 ms
Lookup, governed key	calls/s	171 (171–172)	693 (691–694)	802 (800–806)
	p99	223 ms	57.1 ms	49.9 ms
100-row page	calls/s	596 (595–598)	639 (638–642)	738 (737–740)
	p99	64.8 ms	61.4 ms	52.7 ms
Write	calls/s	682 (679–683)	738 (733–743)	875 (872–881)
	p99	56.9 ms	53.9 ms	45.9 ms

Every range is clear of the others:

- **0.42 over 0.41: +7–9% on plain keys,** from reading the store less; **+304% on a governed key.** Column masking used to trace each statement through its aliases, joins and subqueries on every call, to learn which source column each result column came from. 0.42 works that out once per statement and reuses it. The table check gets the same treatment.
- **The metadata cache adds +16–19% on every call.**
- **The gateway used 1.2 cores for one worker,** and PostgreSQL 0.2–0.4.

One caller at a time

CALL		0.41	0.42	CACHED
Lookup, plain key	p50	1.43 ms (1.43–1.44)	1.38 ms (1.35–1.42)	1.05 ms (1.04–1.06)
	p99 / p99.9	2.1 / 3.0 ms	2.2 / 3.1 ms	1.7 / 2.4 ms
Lookup, governed key	p50	6.00 ms (5.79–6.06)	1.48 ms (1.46–1.51)	1.20 ms (1.16–1.20)
	p99 / p99.9	7.6 / 13.6 ms	2.4 / 3.3 ms	1.9 / 2.5 ms

For a single plain caller, 0.41 and 0.42 overlap: two fewer store reads don't show at this speed. The cache's −24% does. A governed call went from 6 ms to 1.5.

Latency under load

The lookup at fixed offered rates. Capacity, measured first, was 730 (720–797) calls a second for 0.42 and 862 (840–872) with the cache. Each cell is p50 / p99 / p99.9:

OFFERED	0.42	CACHED
25%	2.0 / 3.0 / 11.0 ms	1.5 / 2.3 / 10.5 ms
50%	1.6 / 2.8 / 8.4 ms	1.1 / 2.1 / 9.8 ms
75%	1.5 / 3.8 (3.2-4.1) / 13.5 ms	1.2 / 3.4 (3.3-4.6) / 15.0 ms
90%	1.7 / 11.1 (7.6-12.9) / 21.7 ms	1.2 / 8.7 (6.3-11.6) / 19.0 ms
100%	1.9 / 22.7 (18.3-67.2) / 27.8 ms	1.6 / 17.2 (15.2-23.6) / 21.8 ms
110%	664 / 1,584 / 1,607 ms; 744 of 802 served	305 / 1,276 / 1,296 ms; 893 of 948 served

- **p50 stays flat right up to capacity.** p99 holds at about 4 ms to 75%, then climbs: about 3× at 90%, and 6× at 100%.
- **Past capacity, a queue builds,** and latency reaches seconds within the 20-second run.
- **Real capacity is a little higher than the closed loop says.** At 110%, more was served than "capacity": 744 against 730, and 893 against 862. So each row's real load sits a few percent below its label, and planning at 75% errs on the safe side.
- **The lightest loads show a higher p99.9 than 50%:** at a trickle, an idle core clocks down between requests.

More workers

Workers in one gunicorn, each a process as each container would be, with 64 callers:

WORKERS	LOOKUPS/S: 0.42	LOOKUPS: CACHED	WRITES: 0.42	WRITES: CACHED
1	734 (733-739)	869 (866-871)	732 (729-733)	863 (844-866)
2	1,196 (1,193-1,202)	1,441 (1,431-1,445)	1,188 (1,185-1,190)	1,430 (1,428-1,434)
4	1,716 (1,709-1,722)	2,112 (2,104-2,124)	1,706 (1,701-1,710)	2,087 (2,078-2,094)
8	2,215 (2,127-2,264)	2,727 (2,638-2,801)	2,165 (2,096-2,229)	2,651 (2,552-2,683)

WORKERS	P99 LOOKUPS: 0.42 / CACHED	LOOKUPS/S PER GATEWAY CORE: 0.42 / CACHED	POSTGRESQL CORES: 0.42 / CACHED
1	103 / 88 ms	612 / 736	0.30 / 0.17
2	74 / 59 ms	501 / 622	0.56 / 0.31
4	66 / 69 ms	367 / 463	1.04 / 0.56
8	139 / 92 ms	307 / 377	1.65 / 0.92

- **The cache's lead holds at every size,** +18% to +23%.

- **Throughput grows, but not linearly here.** Calls a second per gateway core fall from 612 at 1 worker to 307 at 8. The eight workers share CPUs 0-7, which are only four physical cores. From four workers on, they share hyperthreads, and a hyperthread is worth far less than a core. Containers on their own cores or hosts would scale better; this run can't show by how much.
- **The cache halves PostgreSQL's work:** 1.65 cores without it at 8 workers, 0.92 with it. Most of PostgreSQL's load was QueryAPIGate's own store, not the queries.

Threads per worker

One worker, the lookup, 32 callers:

THREADS	CALLS/S	P99
1	788 (782-792)	51.7 ms
2	843 (842-851)	47.5 ms
4	777 (772-778)	51.0 ms
8	735 (734-737)	54.5 ms

An earlier post on this blog said queries spend most of their time waiting on the database, so threads do well. That's true for slow queries, and wrong for fast ones:

- **Two threads were the fastest.** One runs Python while the other waits on the database.
- **Eight threads were 13% slower than two, and even slower than one.** That's the Docker image's setting, and the extra threads compete for Python's interpreter lock and the logging module's lock.

Threads are for calls that wait: slow queries, `?stream=true` exports, and `/events` connections that stay open. Match them to your queries; the sizing rule below shows how.

The cold path, every 60 seconds

One 150-second run per key, 8 callers, with each request's duration taken from gunicorn's access log:

KEY	REQUESTS	P50	P99	P99.9	P99.99	SLOWEST
governed	106,741	9.25 ms	14.66 ms	19.2 ms	73.7 ms	146.9 ms
plain	115,314	8.58 ms	13.57 ms	16.9 ms	20.9 ms	26.0 ms

In a typical second, the slowest request took 17 ms. Three seconds of the governed run stand out:

- **t=1, 147 ms:** the very first calls, which read the connection's schema and trace the statement for masking.
- **t=61 and t=121, about 75 ms each:** the same work again, each time QueryAPIGate's copy of the schema expires.

So a governed key pays about 75 ms once a minute, and more on its very first call. Only the requests that arrive during it wait, so it shows at p99.99, not at p99.9. The plain key has nothing to redo, and no spikes.

The access log

ACCESS LOG	CALLS/S	P50	P99
on, the Docker image's default	736 (735-739)	43.0 ms	54.4 ms
off	785 (782-792)	40.2 ms	51.7 ms

gunicorn's access log costs 7%. QueryAPIGate also logs two lines of its own per request, which 0.42 can't turn off.

A database across a network

netem on the database container delays everything PostgreSQL sends by 0.5 ms, about a same-zone cloud round trip. The lookup:

	0.41	0.42	CACHED
One caller: calls/s	133 (132-134)	162 (161-168)	270 (267-273)
One caller: p50	7.57 ms	6.24 ms	3.65 ms
32 callers: calls/s	674 (670-753)	735 (734-741)	855 (852-874)

- **One caller pays for every round trip.** The 0.5 ms hop took a call from 1.4 ms to 6.2 ms. Fewer store reads made 0.42 21% faster than 0.41. The cache made it 67% faster again, and took 41% off a single call, down to 3.65 ms.
- **With 32 callers, the round trips overlap,** and the CPU is the limit again.
- **0.41's range overlaps 0.42's at 32 callers.** One of its five rounds ran at 753 calls a second, on a turbo burst at 4.09 GHz: it was the first run after a `/health` floor, and the rotation gave 0.41 that slot every time. That's a small flaw in our harness, and the fix is on our list. The median is 9% below 0.42's.
- **This is a simulation:** one-way delay, no loss or jitter. We use it below to count round trips, not as a cloud measurement.

Mixed reads and writes, and hot rows

WORKLOAD	CALLS/S	P50	P99
Reads (24 callers) and writes (8) at once	548 (546-549) + 181 (181-182)	43.3 / 43.6 ms	54.4 / 54.9 ms
Writes spread over 10 million accounts	734 (733-736)	43.1 ms	54.4 ms
Writes on 99 hot accounts	738 (735-740)	42.9 ms	53.9 ms

- **Mixing reads and writes changes nothing:** they share the worker's capacity in proportion.
- **Hot rows cost the same as spread-out ones.** One worker sends at most eight writes at once, so PostgreSQL's row locks never queue long enough to show. With many workers or slow transactions, they would.

Where the time goes

Round trips, counted

We turned on PostgreSQL's `log_statement = 'all'`, then made 100 calls after a warm-up:

STATEMENTS PER CALL	0.41	0.42	CACHED
Store, plain key or writer	7: 5 reads, 2 inserts	3: the key, the query, the connection	0.1: history, in batches
Store, governed key	8	4	1: the masking hash key
Data database	4: `BEGIN READ ONLY`, `SET LOCAL statement_timeout`, the query, `ROLLBACK`	4	4
Round trips a call waits for	**9**	**7**	**4**

- **0.41 read the saved query three times.** 0.42 reads it once.
- **History and events are written in batches** of up to 500 by a background thread, so a call doesn't wait for them.
- **A governed call still reads the masking hash key every time,** even with the cache on.
- **Every query takes four round trips to the data database:** a transaction, its time limit, the statement, then the rollback or commit.

Round trips, measured

A statement log can miss traffic. So we also timed single calls at 0, 0.5, 1 and 2 ms of netem delay: the slope of latency against delay is the number of round trips a call waits for. psycopg2 on its own, sending a known 1 or 4 statements, calibrates the method:

	+0 MS	+0.5 MS	+1 MS	+2 MS	SLOPE	ROUND TRIPS (SLOPE ÷ 1.15)
psycopg2, 1 statement	0.03	0.63	1.19	2.33	1.15	1
psycopg2, a query's 4 statements	0.06	2.44	4.68	9.36	4.60	4.0
0.41	1.42	7.60	12.47	22.66	10.46	9.1
0.42	1.43	6.24	10.01	18.75	8.58	7.5
Cached	1.05	3.70	5.84	10.59	4.72	4.1

Under netem, a round trip costs 1.15 ms per millisecond of delay, the same for bare psycopg2 at 1 statement and at 4. That's netem's per-packet overhead. Divided by it, the measured round trips match the counted ones: 9.1 against 9, 7.5 against 7, and 4.1 against 4. The statement log missed nothing.

The CPU

A py-spy profile, 200 samples a second across the worker's threads, during one 30-second run of the lookup at 32 callers:

WHERE THE SAMPLES FALL	0.42	CACHED
Store layer: queries, decoding rows, checks	28.2%	13.4%
Logging: the access log and QueryAPIGate's own lines	18.4%	20.4%
gunicorn, sockets, HTTP	16.8%	21.0%
Data query: the driver and the pool	12.3%	15.4%
Flask and Werkzeug	9.9%	11.9%
QueryAPIGate's request handling	4.6%	5.6%
History writer thread	3.0%	3.7%
Authentication, limits, quotas, metrics	2.5%	2.8%
SQL checks, parameters, masking	1.7%	1.9%
JSON	1.3%	2.6%
Other	1.3%	1.3%

The busiest single frames say more than the categories:

- **8-10% is threads waiting for the logging module's lock.** Eight threads write log lines through one handler.
- **7-10% is one safety check,** which makes sure a connection doesn't point at QueryAPIGate's own SQLite file. It resolves file paths, several system calls, on every call - even for a PostgreSQL connection, which has no file to point at.
- **17% is the three store reads in 0.42,** gone with the cache.
- **3% is gunicorn's per-request worker heartbeat.**

Sizing

A worker runs out of one of two things first:

- **CPU.** Plan on about 550 fast calls a second per worker, or 650 with the metadata cache. That's 75% of measured capacity, where p99 stays around 4 ms. Every millisecond of Python a call costs beyond this lookup lowers it.
- **Waiting.** A worker holds at most as many calls at once as it has threads, and each call waits for its query plus its round trips: 7 in 0.42, or 4 cached, at the network's round-trip time each. So a worker serves at most threads ÷ (query time + round trips × round-trip time) calls a second.

Workers needed = the larger of (calls a second ÷ the CPU figure) and (calls a second ÷ the waiting figure), plus one you can afford to lose.

Two examples, with a 0.5 ms network round trip and 8 threads:

CALLS	CPU FIGURE	WAITING FIGURE	WHAT LIMITS A WORKER
A 0.03 ms lookup, like ours	550 a second	8 ÷ (0.03 + 7 × 0.5 ms) ≈ 2,270 a second	CPU - so fewer threads would do
A 20 ms report query	550 a second, or fewer	8 ÷ (20 + 3.5 ms) ≈ 340 a second	Waiting - so more threads would help

These are planning figures from one machine and one query shape. Before you commit to a number, measure your own queries at 75% of what you plan to serve.

What to do with this

- **Size with the rule above,** then measure your own queries, which will usually cost more than ours.
- **Turn on** `QUERYAPIGATE_METADATA_CACHE_TTL` if you can accept a few seconds' delay for a change made on another instance to reach this one. It caches the rows, never a decision: whether a key is active, expired or used from an allowed address is still checked on every call, and a change made through the same server clears the cache at once. In return, a worker serves 16-23% more, PostgreSQL does half the work, and across a network a single call takes 41% less time.
- **Match threads to your queries:** 2 for fast lookups, which beats the default 8 by 15% here. Keep 8 or more for slow queries, exports and `/events`.
- **Turn off gunicorn's access log** if a proxy in front of it already logs requests: +7%.
- **Scale with processes on their own cores.** A hyperthread gave about half a core here.

What's next, in 0.43

The profile and the round-trip count turned the benchmark into a to-do list:

1. **Fewer round trips per query.** Set the read-only mode and the time limit once per pooled connection instead of in every transaction, which takes a query from 4 round trips to 1. At a 0.5 ms round trip, that's about 1.7 ms off every call: from 6.2 to about 4.5 ms, and from 3.7 to about 2 ms cached.
2. **Skip the file-path safety check for server databases.** Only SQLite, DuckDB and H2 open a file, so this saves 7-10% of CPU on every PostgreSQL, MySQL, ClickHouse and Trino call.
3. **Cache the masking hash key,** which is read on every governed call and almost never changes.
4. **Cheaper logging:** a setting to skip the per-request lines, and no single lock for every thread to wait on.
5. **Refresh the schema and masking trace in the background,** so no governed call pays the 75 ms each minute.

What we got wrong first

This is our third write-up of these numbers. The first two had problems that reviewers caught.

The first had two errors:

- **It compared single runs on a laptop that, it turned out, was on battery** and a power-saving profile, with nothing pinned. It reported about half the throughput this post does - 368 lookups a second where we now measure 743, with the same code. Three things changed at once since: power, core pinning, and taking Docker's proxy out of the path. We didn't test them one at a time, so we can't say which was worth how much. Power is likely most of it, because `/health`, which never touches PostgreSQL, doubled as well.
- **It miscounted the store reads.** It said a call's store transactions went from about 5.5 to 2.6. They actually went from 7 statements to 3. PostgreSQL can hold back an idle session's statistics for up to 10 seconds, and we read them 2 seconds after each run.

The second fixed the method - pinning, interleaving, ranges, open-loop load - and a review rated it 4.1 out of 5. It still had problems, each fixed here:

- **Some 3-round comparisons,** which, as above, are wrong 10% of the time. Every one was rerun with 5.
- **A threads result that cleared by 3 calls a second.** With 5 rounds, it's clear by 50.
- **Two overstatements.** "Halved single-call latency" is now "−41%". "60-140 ms spikes every minute" is now "about 75 ms every minute, more on the first call".
- **A round-trip count that didn't explain its own network result.** The calibrated slope test above settles it.
- **Latency percentages measured against too low a "capacity",** a clock range recorded but never used, and the load generator's CPU not recorded at all. All three are covered above.

What this doesn't cover

- **One machine.** The gateway, the database and the load generator share one memory bus and one cooling system, on separate cores. Four physical cores for the gateway limit what scale-out can show.
- **No TLS, load balancer or Redis in the path.** Each would add to the cost of a call.
- **A simulated network:** one-way delay, without jitter or loss.
- **Uniform random keys.** Real traffic is skewed. We tested hot rows for writes, but not a skewed read pattern.
- **Small results only:** one row, or a page of 100. Not large results, streaming exports, the MCP server, or other databases.
- **p99.99 only from the 150-second runs.** Elsewhere, 30-second runs give about 20,000 samples, enough for p99.9.
- **A profiler that counts threads waiting for Python's lock as busy.** So the shares say where time goes, not exactly how much a fix would save.
- **No soak or failure tests:** hours at 75% load, or losing PostgreSQL or a worker mid-run.

Check it yourself

The reference write-up, with every table, is Performance, v3. The raw results, one JSON line per run, are in `benchmarks/results/`. The harness is in the repository:

```shell
pip install -e ".[server,postgres,flow]" py-spy
python benchmarks/oltp.py --out /tmp/qag-oltp setup all teardown     # about 3 hours 45 minutes
python benchmarks/oltp_report.py /tmp/qag-oltp/results.jsonl         # every table, median and range per
cell
```

It starts its own PostgreSQL with this schema, pins each part to its own cores (choose yours with `--gateway-cpus`, `--db-cpus` and `--load-cpus`), compares two git refs side by side (`--base` and `--head`), and removes everything when it's done. `--smoke` checks the setup in about ten minutes.

Plug the laptop in first. Then point it at your own machine and release, and tell us what you find.